

Предложения по улучшению подпроекта `java`

Проект fullWeb (Query) — Kotlin Multiplatform fullstack. Анализ модуля `:java` (Ktor/Netty backend). Состав: `Server.kt`, `Routes.kt`, `PgSQL.kt` (~300 строк), `FromJsonString.kt`, `build.gradle.kts`. Тестов в модуле нет. Приоритеты: **P0 — критично**, **P1 — важно**, **P2 — улучшение качества**.

P0 — Безопасность (исправлять в первую очередь)

1. **RCE:** `GET /com/{command}` (`Routes.kt:36-47`) — выполняет произвольные OS-команды через `Runtime.exec()` без какой-либо аутентификации. Рекомендация: удалить эндпоинт вовсе или закрыть (только localhost + проверка секрета + жёсткий allowlist команд).
2. **SQL-инъекции** (`PgSQL.kt`):
 - `checkInjection` (`:161-164`) — чёрный список из 5 слов (`OR`, `;`, `CREATE`, `INSERT`, `DELETE`, `SELECT`), обходится тривиально; при этом блокирует легитимные `SELECT`-представления. Заменить на allowlist имени представления (`^[A-Za-z0-9_.$]+`) + параметризацию значений через `PreparedStatement`.
 - `where` из URL напрямую подставляется в `where 0=0 $where` (`:125`) — главный вектор. Ограничить разрешённые символы/операторы либо параметризовать.
 - `insertParameterValue` (`shared/CommonVars.kt:76-87`) — сырая подстановка `{key}` без экранирования.
 - `replaceEscape` (`:293-302`) — экранирует только одинарные кавычки (Postgres); для BigQuery (`QUOTE_THREE`) значения не экранируются вовсе.
 - `whereSQL / formSQLQuery` (`:183-237`) — `UPDATE/DELETE/INSERT` строятся конкатенацией из пользовательского JSON.
3. **CORS** `anyHost()` (`Server.kt:36`) — разрешает любой origin. Ограничить список origin.
4. **Открытые эндпоинты без аутентификации:** `POST /jsr223` (`Routes.kt:18`), `GET /{key}/` (`:48-54`) — удалить или закрыть.
5. **Проверка пароля** (`Routes.kt:56-62`) — сравнение строки в открытом виде, без rate-limiting. Использовать `MessageDigest.isEqual` + ограничение попыток.

P1 — Потокобезопасность (критично для многопоточного Ktor)

6. **Гонки в `FromJsonString.kt`:** глобальные `var QUOTE_TYPE` и `var mappingFun` (`:10-11`) мутируются на каждый запрос — параллельные запросы с разными `quoteType` перезапишут друг друга. Решение: передавать `quoteType` параметром/через `ThreadLocal`.
7. **Гонки в `PgSQL.kt`:** глобальные `var baseName/url/user/pass` (`:12-15`) перезаписываются при каждом подключении; `var connectionError` — тоже общая. Перенести в локальные переменные/структуру состояния.
8. `connectionsMap` (`:11`) — `mutableMapOf` без синхронизации → `ConcurrentHashMap` + атомарная инициализация (`computeIfAbsent`).
9. **Глобальный `var message`** (`Routes.kt:10`) — убрать, возвращать результат из функции.

P1 — Управление подключениями к БД

10. **Нет пула соединений:** `DriverManager.getConnection`, одно соединение на базу (`:52`). Перейти на **HikariCP** — пул, валидация (`isValid`), переподключение, таймауты.
11. **Сертификат на Cloud Run:** `setProperties` пишет `./$baseName.crt` в рабочую директорию (`:94-97`) — на Cloud Run ФС только для чтения. Писать в `System.getProperty("java.io.tmpdir")`.
12. **Ресурсы:** `Statement / ResultSet` местами не закрываются (`pgsqlJsn`), `connectionError` не сбрасывается. Использовать `use {}`.
13. **Парсинг `connection string`** (`splitURL`, `:23-33`) вручную по `split` — заменить на `java.net.URI`.
14. **BigQuery:** захардкоженный `projectId = "vpn_gra"/"vpn-gra"` (`:261, 279`) — брать из env; `ROW_TO_JSON` →

TO_JSON_STRING заменой строки (:132) хрупко; блокирующий query() — задать таймаут/PageSize.

P2 — Архитектура и качество кода

15. **Разделить PostgreSQL.kt** на классы/файлы: PgConnectionManager, BigQueryClient, SqlBuilder, SqlInjectionGuard, ViewRepository.
16. **Разделить Routes.kt** — маршруты вынести в отдельные extension-функции Route.xxx(); скрытое глобальное состояние заменить контекстом приложения (Application.attributes).
17. **Тесты:** добавить src/jvmTest (kotlin-test) для SqlBuilder / checkInjection / fromJsonString / splitURL / UPDATE-DELETE-INSERT.
18. **Логирование:** заменить println на SLF4J (LoggerFactory), добавить logback.xml, CallLogging. Не логировать SQL с секретами и полные параметры.
19. **Мёртвый код:** закомментированные блоки (Routes.kt:15–17, 109–118) — почистить.
20. **Health-check:** добавить GET /health и/или /ready для Cloud Run/Koyeb + graceful shutdown.
21. **Rate limiting** для чувствительных маршрутов (в Ktor 3 есть встроенный RateLimit).

P2 — Сборка (build.gradle.kts)

22. **myProp()** (:139–159) — побочный эффект на этапе конфигурации Gradle: перезаписывает config.properties, запускает git при каждой сборке. Перенести в doLast /отдельную задачу.
23. **String.runCommand** (:116–128) — разбиение команды regex-ом по пробелам, ломается на путях с пробелами → ProcessBuilder с массивом аргументов.
24. **Версии и группа:** group = "org.example" — заменить на реальный; ktorVersion / serializationVersion дублируются между java и shared → вынести в version catalog (gradle/libs.versions.toml).
25. **Чистка импортов** в build.gradle.kts (неиспользуемые DuplicatesStrategy, ProcessResources, IOException, FileOutputStream).

Порядок реализации

1. Удалить/закрыть GET /com/{command}, /{key}/, /jsr223; ограничить CORS.
2. Починить потокобезопасность (FromJsonString, глобальные var в PostgreSQL / Routes, connectionsMap).
3. Заменить checkInjection на allowlist + параметризацию; безопасное экранирование значений.
4. HikariCP + фикс сертификата в tmpdir + закрытие ресурсов.
5. Рефакторинг PostgreSQL.kt / Routes.kt, /health, логирование.
6. Тесты + чистка build-скрипта.